Bit Manipulation Study Guide

1. Masking

What is Bit Masking?

Bit masking is a technique used to manipulate individual bits or groups of bits within a binary number. A mask is a binary pattern that is used with bitwise operators to turn specific bits on, off, or flip them.

Common Bitwise Operators

- **AND (&)**: Returns 1 if both bits are 1
- **OR (|)**: Returns 1 if at least one bit is 1
- **XOR (^)**: Returns 1 if bits are different
- **NOT (~)**: Flips all bits
- **Left Shift (<<)**: Shifts bits to the left
- **Right Shift (>>)**: Shifts bits to the right

Types of Masking Operations

Setting a Bit (Turn ON)

To set the i-th bit, use OR operation with a mask that has 1 at the i-th position.

```java
public class BitMasking {
    // Set the i-th bit (0-indexed from right)
    public static int setBit(int num, int i) {
        return num | (1 << i);
    }

    public static void main(String[] args) {
        int number = 5; // Binary: 101
        int result = setBit(number, 1); // Set bit at position 1
        System.out.println("Original: " + number + " (" + Integer.toBinaryString(number) + ")")
        System.out.println("After setting bit 1: " + result + " (" + Integer.toBinaryString(res
        // Output: 7 (111)
    }
}
```

Clearing a Bit (Turn OFF)

To clear the i-th bit, use AND operation with a mask that has 0 at the i-th position.

```java
public class BitMasking {
    // Clear the i-th bit (0-indexed from right)
    public static int clearBit(int num, int i) {
        return num & ~(1 << i);
    }

    public static void main(String[] args) {
        int number = 7; // Binary: 111
        int result = clearBit(number, 1); // Clear bit at position 1
        System.out.println("Original: " + number + " (" + Integer.toBinaryString(number) + ")")
        System.out.println("After clearing bit 1: " + result + " (" + Integer.toBinaryString(re
        // Output: 5 (101)
    }
}
```

Toggling a Bit (Flip)

To toggle the i-th bit, use XOR operation with a mask that has 1 at the i-th position.

```java
public class BitMasking {
    // Toggle the i-th bit (0-indexed from right)
    public static int toggleBit(int num, int i) {
        return num ^ (1 << i);
    }

    public static void main(String[] args) {
        int number = 5; // Binary: 101
        int result = toggleBit(number, 2); // Toggle bit at position 2
        System.out.println("Original: " + number + " (" + Integer.toBinaryString(number) + ")")
        System.out.println("After toggling bit 2: " + result + " (" + Integer.toBinaryString(re
        // Output: 1 (001)
    }
}
```

Checking a Bit

To check if the i-th bit is set, use AND operation with a mask.

```java
public class BitMasking {
    // Check if i-th bit is set (0-indexed from right)
    public static boolean isBitSet(int num, int i) {
        return (num & (1 << i)) != 0;
    }

    public static void main(String[] args) {
        int number = 5; // Binary: 101
        System.out.println("Number: " + number + " (" + Integer.toBinaryString(number) + ")");
        System.out.println("Bit 0 is set: " + isBitSet(number, 0)); // true
        System.out.println("Bit 1 is set: " + isBitSet(number, 1)); // false
        System.out.println("Bit 2 is set: " + isBitSet(number, 2)); // true
    }
}
```

2. Kernighan's Algorithm

What is Kernighan's Algorithm?

Kernighan's Algorithm is an efficient method to count the number of set bits (1s) in the binary representation of a number. It works by repeatedly clearing the lowest set bit until the number becomes zero.

How It Works

The key insight is that `n & (n-1)` always clears the lowest set bit in n.

Example:

- n = 12 (binary: 1100)
- n-1 = 11 (binary: 1011)
- n & (n-1) = 1100 & 1011 = 1000 (cleared the lowest set bit)

Implementation

```java
public class KernighansAlgorithm {
    // Count number of set bits using Kernighan's algorithm
    public static int countSetBits(int n) {
        int count = 0;
        while (n != 0) {
            n = n & (n - 1); // Clear the lowest set bit
            count++;
        }
        return count;
    }

    public static void main(String[] args) {
        int number = 12; // Binary: 1100
        System.out.println("Number: " + number + " (" + Integer.toBinaryString(number) + ")");
        System.out.println("Number of set bits: " + countSetBits(number));

        // Step by step visualization
        System.out.println("\nStep by step:");
        int n = 12;
        int step = 1;
        while (n != 0) {
            System.out.println("Step " + step + ": " + n + " (" + Integer.toBinaryString(n) + "
            n = n & (n - 1);
            step++;
        }
    }
}
```

Time Complexity

- **Time Complexity:** O(number of set bits) instead of O(log n)
- **Space Complexity:** O(1)

Practical Applications

- Counting population (number of 1s) in binary representation
- Checking if a number is a power of 2: `(n & (n-1)) == 0`
- Finding the next power of 2

```java
public class KernighansApplications {
    // Check if number is power of 2
    public static boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }

    // Find next power of 2
    public static int nextPowerOfTwo(int n) {
        if (n <= 1) return 1;
        n--;
        n |= n >> 1;
        n |= n >> 2;
        n |= n >> 4;
        n |= n >> 8;
        n |= n >> 16;
        return n + 1;
    }

    public static void main(String[] args) {
        System.out.println("Is 8 power of 2? " + isPowerOfTwo(8)); // true
        System.out.println("Is 12 power of 2? " + isPowerOfTwo(12)); // false
        System.out.println("Next power of 2 after 10: " + nextPowerOfTwo(10)); // 16
    }
}
```

3. Practice Questions on Bit Manipulation

Problem 1: Single Number

Problem: Given an array where every element appears twice except one, find the single number.

```java
public class SingleNumber {
    public static int findSingle(int[] nums) {
        int result = 0;
        for (int num : nums) {
            result ^= num; // XOR all numbers
        }
        return result;
    }

    public static void main(String[] args) {
        int[] arr = {2, 3, 5, 4, 5, 3, 4};
        System.out.println("Single number: " + findSingle(arr)); // Output: 2
    }
}
```

Problem 2: Count Set Bits in Range

Problem: Count total set bits in all numbers from 1 to n.

```java
public class CountSetBitsRange {
    public static int countSetBitsInRange(int n) {
        int count = 0;
        for (int i = 1; i <= n; i++) {
            count += countSetBits(i);
        }
        return count;
    }

    // Using Kernighan's algorithm
    private static int countSetBits(int n) {
        int count = 0;
        while (n != 0) {
            n = n & (n - 1);
            count++;
        }
        return count;
    }

    public static void main(String[] args) {
        int n = 5;
        System.out.println("Total set bits from 1 to " + n + ": " + countSetBitsInRange(n));
        // Numbers: 1(1), 2(10), 3(11), 4(100), 5(101)
        // Set bits: 1 + 1 + 2 + 1 + 2 = 7
    }
}
```

Problem 3: Bit Manipulation for Subsets

Problem: Generate all possible subsets of a set using bit manipulation.

```java
import java.util.*;

public class GenerateSubsets {
    public static List<List<Integer>> generateSubsets(int[] nums) {
        List<List<Integer>> subsets = new ArrayList<>();
        int n = nums.length;

        // Generate all possible combinations using bit masking
        for (int mask = 0; mask < (1 << n); mask++) {
            List<Integer> subset = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                // Check if i-th bit is set
                if ((mask & (1 << i)) != 0) {
                    subset.add(nums[i]);
                }
            }
            subsets.add(subset);
        }
        return subsets;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        List<List<Integer>> result = generateSubsets(arr);
        System.out.println("All subsets:");
        for (List<Integer> subset : result) {
            System.out.println(subset);
        }
    }
}
```

Problem 4: Find Missing Number

Problem: Given an array containing n distinct numbers from 0 to n, find the missing number.

```java
public class FindMissingNumber {
    public static int findMissing(int[] nums) {
        int n = nums.length;
        int xor1 = 0, xor2 = 0;

        // XOR all numbers from 0 to n
        for (int i = 0; i <= n; i++) {
            xor1 ^= i;
        }

        // XOR all numbers in array
        for (int num : nums) {
            xor2 ^= num;
        }

        return xor1 ^ xor2; // Missing number
    }

    public static void main(String[] args) {
        int[] arr = {3, 0, 1}; // Missing: 2
        System.out.println("Missing number: " + findMissing(arr));
    }
}
```

Multiple Choice Quiz

Question 1: What does the expression `n & (n-1)` do? a) Sets the lowest bit b) Clears the highest set bit c) Clears the lowest set bit d) Toggles all bits

Answer: c) Clears the lowest set bit

Question 2: Which bitwise operator is used to set a specific bit? a) AND (&) b) OR (|) c) XOR (^) d) NOT (~)

Answer: b) OR (|)

Question 3: What is the time complexity of Kernighan's algorithm for counting set bits? a) O(log n) b) O(n) c) O(number of set bits) d) O(1)

Answer: c) O(number of set bits)

Question 4: To check if a number is a power of 2, which condition should be true? a) `n & (n+1) == 0` b) `n & (n-1) == 0` c) `n | (n-1) == 0` d) `n ^ (n-1) == 0`

Answer: b) `n & (n-1) == 0`

Question 5: What will be the result of `5 ^ 3 ^ 5`? a) 0 b) 3 c) 5 d) 8

Answer: b) 3 (XOR is commutative, so 5 ^ 5 = 0, and 0 ^ 3 = 3)

Practice Coding Questions

Question 1: Write a Java program to swap two numbers without using a temporary variable, using only bitwise XOR operations.

Question 2: Given an integer array where every element appears three times except one element which appears only once, find that single element using bit manipulation.

Question 3: Write a program to reverse the bits of a given 32-bit unsigned integer. For example, if the input is 43261596 (binary: 00000010100101000001111010011100), return 964176192 (binary: 00111001011110000010100101000000).

Question 4: Find the two non-repeating elements in an array where every other element repeats exactly twice. Use bit manipulation techniques.

Question 5: Write a program to count the number of bits that need to be flipped to convert integer A to integer B using bitwise operations.